

CS 421 Examination

Functional Programming Paradigms

Instructions:

- Answer all questions.
- For Questions 1–4, choose the best option.
- For Questions 5–8, write brief but precise answers.

1. Which property defines a pure function in functional programming?
 - (A) It always returns the same output for the same input and has no side effects
 - (B) It can modify global state but returns consistent results
 - (C) It uses recursion instead of loops
 - (D) It operates only on primitive data types
2. In the context of lazy evaluation, when are expressions evaluated?
 - (A) Immediately when defined
 - (B) Only when their values are needed
 - (C) At compile time through optimization
 - (D) Never, if they are not used in the final output
3. What is a monad in functional programming?
 - (A) A design pattern for implementing loops
 - (B) A type class that provides a way to chain operations while abstracting computational context
 - (C) A data structure for storing immutable collections
 - (D) A method for parallel computation
4. In functional programming, what does the term “referential transparency” mean?
 - (A) Functions can be replaced by their return values without changing program behavior
 - (B) Variables can be reassigned multiple times
 - (C) Functions must explicitly state all dependencies
 - (D) All data structures are transparent to the compiler

5. Explain the concept of higher-order functions and provide examples of how they promote code reusability. How do functions like `map`, `filter`, and `reduce` demonstrate this principle?
6. Describe the difference between eager and lazy evaluation strategies. What are the advantages and potential pitfalls of lazy evaluation? Provide an example where lazy evaluation enables working with infinite data structures.
7. Explain the concept of currying and partial application in functional programming. How do these techniques differ, and what practical benefits do they provide in composing functions?
8. Discuss the concept of immutability in functional programming. Why is immutability considered important, and how does it facilitate reasoning about concurrent programs? What are the trade-offs in terms of performance?